PERSONAL INFORMATION MANAGEMENT (PIM) SYSTEM

Developer Manual

Document	Developer Manual
Version	1.4
Date	8 November 2026
Group	Group [number]
Status	Final

Group members

Name	Student ID
[Member 1 name]	[Student ID]
[Member 2 name]	[Student ID]
[Member 3 name]	[Student ID]
[Member 4 name]	[Student ID]

Contents

1 Platform and tools 3

2 Building the project 3

3 Source layout 3

4 Opening and running the project 4

5 Debugging 5

6 Running the tests 5

7 Measuring line coverage 6

8 The commit gate 6

9 Building the documents 6

1 Platform and tools

This manual covers **macOS** only.

Item	Version or choice
Operating system	macOS (developed and tested on macOS 27)
Language	Python 3.12 or newer. The project was developed with CPython 3.12.4 from python.org; the test suite also passes on Python 3.15.
Libraries	The Python standard library only. Do not add pip packages.
IDE	Visual Studio Code with Microsoft's Python extension (`ms-python.python`), which includes the Python Debugger
Terminal	Terminal or iTerm, or the integrated terminal of VS Code

Always type `python3` in the commands below. macOS has no `python` command by default, and its built-in `/usr/bin/python3` may be older than 3.12; check with `python3 --version`.

The model reads the time zone `Asia/Hong_Kong` from the time-zone database that macOS provides, through the standard `zoneinfo` module. On a computer without that database, it falls back to a fixed UTC+8 zone named HKT, so the program still starts.

2 Building the project

There is nothing to compile and no build step: Python runs the source files directly, and the project has no dependencies to install. To check that every module compiles, run this at the root of the source code:

```
python3 -m compileall -q pim.py model view controller
```

3 Source layout

Path	Contents
`pim.py`	Composition root: creates `PIM()`, `App(pim)`, and `Terminal(app)`, then calls `run()`
`model/`	The Working Collection, the PIR types, alarms, Search Criteria, and the `.pim` JSON format. This is the unit-test surface; it must not import `view` or `controller`.
`controller/`	`App`, which turns one completed user action into model calls, and `errors.message_for`
`view/`	The terminal interface: `Terminal` (prompts and the line interface), `CursesUI` (the full-screen interface), and helpers for layout, selectors, the calendar, the folder browser, keys, colours, and text width

Path	Contents
`tests/unit/`	Unit tests of `model` (100 % line coverage), `controller.App`, and the View classes with fake collaborators
`tests/integration/`	`App` against a real `PIM`, and `Terminal` against `App` and `PIM`
`tests/e2e/`	Scripted terminal sessions through `Terminal.run()`
`tests/fixture.py`	The shared six-PIR test collection
`tests/coverage_report.py`	Writes `docs/04-unit-tests/coverage.txt`, the line coverage of `model/`
`hooks/pre-commit`	Git hook that refuses a commit unless all tests pass
`docs/01-requirements/`	SRS source
`docs/02-design/`	Design document source
`docs/03-implementation/`	This manual, the user manual, and the requirements coverage report
`docs/04-unit-tests/`	Line-coverage report source
`docs/05-presentation/`	Slide deck source and build script
`docs/group.yaml`	Group details shared by title pages and slides
`docs/_diagrams/`	The diagram sources of the design document
`docs/_typeset/`	The page design of the PDFs, which `docs/build.sh` builds
PDF files at the root	The typeset documents, including the requirements coverage report (`REQUIREMENTS.pdf`) and the line-coverage report (`COVERAGE_REPORT.pdf`)

The design document describes every class and how the packages depend on each other.

4 Opening and running the project

In VS Code. Choose *File* → *Open Folder…* and open the folder that contains `pim.py`. Then run *Python: Select Interpreter* from the Command Palette and pick a Python 3.12 (or newer) interpreter.

From a terminal, at the root of the source code:

```
python3 pim.py
```

When both standard input and standard output are an interactive terminal, the program opens the full-screen `curses` interface. It uses the plain line interface instead when:

- input or output is redirected, as in the tests;
- the environment variable `PIM_NO_CURSES` is set to any value other than an empty string, `0`, `false`, or `no`;
- the `curses` module cannot be imported, or fails to start (for example with `TERM=dumb`).

To force the line interface:

```
PIM_NO_CURSES=1 python3 pim.py
```

On a terminal with at least 256 colours, the full-screen interface uses its own dark colour scheme; with 8 colours it uses reverse video for the title and the selection, and red and amber for alarms.

5 Debugging

VS Code debugger. Open `pim.py` and choose *Run and Debug → Python Debugger: Debug Python File*. The program runs in the integrated terminal, which is interactive, so the full-screen interface appears there, and breakpoints in `model/`, `controller/`, or `view/` stop as usual.

To step through the code without the full-screen interface, create a launch configuration: open the Run and Debug view, click *create a launch.json file*, and choose *Python Debugger → Python File*. Then add an `env` entry to the configuration in `.vscode/launch.json`:

```
"env": { "PIM_NO_CURSES": "1" }
```

Standard-library debugger. The `pdb` prompt and the full-screen interface cannot share the terminal, so start the program in the line interface:

```
PIM_NO_CURSES=1 python3 -m pdb pim.py
```

To debug a model rule, it is usually quicker to run one unit test under `pdb`:

```
python3 -m pdb -m unittest tests.unit.test_pim
```

6 Running the tests

All three layers, from the root of the source code:

```
python3 -m unittest
```

One layer at a time:

```
python3 -m unittest discover -s tests/unit -t .
python3 -m unittest discover -s tests/integration -t .
python3 -m unittest discover -s tests/e2e -t .
```

The tests use only `unittest`. They inject their own input and output streams and the current time, so they never open the full-screen interface and never depend on the clock. Do not call `curses.initscr` from a test.

7 Measuring line coverage

```
python3 tests/coverage_report.py
```

The script runs every test in `tests/unit` under the standard `trace` module and writes `docs/04-unit-tests/coverage.txt`, the source of `COVERAGE_REPORT.pdf` at the root of the source code. A line counts when a Python statement starts on it; docstrings, comments, and the continuation lines of a statement are not counted, and `model/__init__.py`, which only re-exports names, is excluded. The exit status is 1 if a unit test fails or if any countable line of `model/` was not executed. With `--check`, as used by the commit hook, the script follows the same rules but rewrites `coverage.txt` only when coverage is 100 %.

The totals can differ slightly from those of the third-party `coverage.py`, which this project does not use.

8 The commit gate

Enable the hook once per clone:

```
git config core.hooksPath hooks
```

Every `git commit` then runs the unit tests with the 100 % coverage check, then the integration tests, then the end-to-end tests, and stops at the first failure. `git commit --no-verify` skips the hook.

9 Building the documents

The PDF versions of the SRS, the design document, the two manuals, the requirements coverage report, and the coverage report are typeset from their Markdown sources. This step is for maintainers only and is not needed to run or test the system. It needs pandoc 3, Mermaid CLI (`mmdc`), Google Chrome, and either MacTeX (XeLaTeX) or Tectonic with Source Sans 3:

```
sh docs/build.sh
```

Every PDF is written to the root of the source code. The group number and member names on the title pages come from `docs/group.yaml`.